

Machine Learning

Volker Roth

Department of Mathematics & Computer Science
University of Basel

Chapter 4: Classification

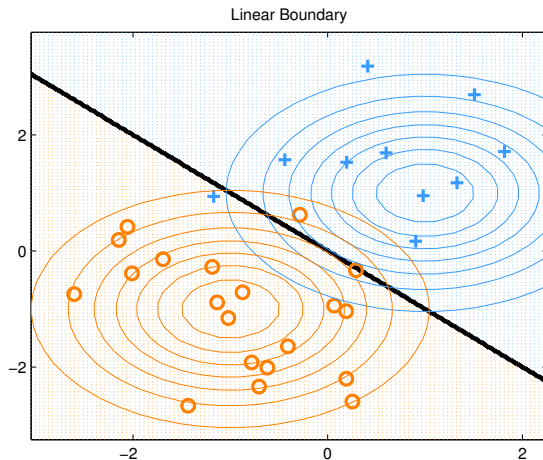


Fig 4.5 in K. Murphy: Machine Learning. MIT Press 2012

Bayesian Decision Theory

- Assign observed $\mathbf{x} \in \mathbb{R}^d$ into one of k classes. A **classifier** is a mapping that assigns labels to observations

$$f_{\alpha} : \mathbf{x} \rightarrow \{1, \dots, k\}.$$

- For any observation $\mathbf{x}$ there exists a set of k **possible actions** α_i , i.e. k different assignments of labels.
- The **loss** L incurred for taking action α_i when the true label is j is denoted by a loss matrix $L_{ij} = L(\alpha_i | c = j)$.
- “Natural” **0 – 1 loss function** $\rightsquigarrow$ **counting misclassifications**:

$$L_{ij} = 1 - \delta_{ij}, \text{ where } \delta_{ij} = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{otherwise.} \end{cases}$$

Bayesian Decision Theory (cont'd)

- A classifier is trained on a set of **observed pairs**, drawn from a **true but unknown “state of the nature”**, formalized by the joint probability $p(\mathbf{x}, c)$: $\{(\mathbf{x}_1, c_1), \dots, (\mathbf{x}_n, c_n)\} \stackrel{\text{i.i.d.}}{\sim} p(\mathbf{x}, c) = p(c|\mathbf{x})p(\mathbf{x})$
- We assume that $\mathbf{x}$ can be observed for every object, but the true class labels are unknown (i.e. these are unobservable states of nature).
- The probability that a given $\mathbf{x}$ is member of class c_j , i.e. the posterior probability of membership in class j , is obtained via the **Bayes rule**:

$$P(c_j|\mathbf{x}) = \frac{\text{Given the label, observation is generated } p(\mathbf{x}|c=j)}{p(\mathbf{x})} \quad \text{Nature picks a label first } P(c=j),$$

where

$$p(\mathbf{x}) = \sum_{j=1}^k p(\mathbf{x}|c=j)P(c=j).$$

- Given an observation $\mathbf{x}$, the expected loss associated with choosing action α_i (the **conditional risk** or **posterior expected loss**) is

$$R(f_{\alpha_i}|\mathbf{x}) = \sum_{j=1}^k L_{ij}P(c_j|\mathbf{x}) \stackrel{(\text{if } L_{ij}=1-\delta_{ij})}{=} \sum_{j \neq i} P(c_j|\mathbf{x}) = 1 - P(c_i|\mathbf{x}).$$

Bayesian Decision Theory (cont'd)

- **Goal:** minimize the **Bayes risk** a.k.a. **integrated risk** of the classifier f_α :

$$R(f_\alpha) = \int_{\mathbf{R}^d} R(f_\alpha(\mathbf{x})|\mathbf{x})p(\mathbf{x}) d\mathbf{x} = \mathbb{E}_{p(\mathbf{x})}\mathbb{E}_{p(c|\mathbf{x})}\text{loss}(f_\alpha(\mathbf{x}), c).$$

- If $f_\alpha(\mathbf{x})$ minimizes the conditional risk $R(f_\alpha(\mathbf{x})|\mathbf{x})$ for every $\mathbf{x}$, the overall risk will be minimized as well.
- This is achieved by the **Bayes optimal classifier** which chooses the mapping

$$f(\mathbf{x}) = \underset{i}{\operatorname{argmin}} \sum_{j=1}^k L_{ij} P(c = j|\mathbf{x}).$$

- For 0 – 1 loss, this reduces to classifying $\mathbf{x}$ to the class **with highest posterior probability**:

$$f(\mathbf{x}) = \underset{i}{\operatorname{argmax}} P(c = i|\mathbf{x}).$$

Bayesian Decision Theory (cont'd)

- **Simplification:** only 2 classes: c is **Bernoulli RV**.
- Bayes optimal classifier is defined by the **zero crossings** of the **Bayes optimal discriminant function**

$$G(\mathbf{x}) = P(c_1|\mathbf{x}) - P(c_2|\mathbf{x}), \text{ or } g(\mathbf{x}) = \log \frac{P(c_1|\mathbf{x})}{P(c_2|\mathbf{x})}.$$

- Problem: direct approximation of G would require the knowledge of the Bayes optimal discriminant.
- Define a **parametrized family of classifiers** $\mathcal{F}_{\mathbf{w}}$ from which we can choose one (or more) function(s) by some **inference mechanism**.
- One such family: linear discriminant functions $g(\mathbf{x}; \mathbf{w}) = w_0 + \mathbf{w}^t \mathbf{x}$.
- Two-category case: Decide c_1 if $g(\mathbf{x}; \mathbf{w}) > 0$ and c_2 if $g(\mathbf{x}; \mathbf{w}) < 0$.
Eq. $g(\mathbf{x}; \mathbf{w}) = 0$ defines the **decision surface**.
- The hyperplane divides the feature space into **half-spaces** $\mathcal{R}_1$ ("positive side") and $\mathcal{R}_2$ ("negative side").

Decision Hyperplanes

- $g(\mathbf{x}; \mathbf{w})$ defines distance r from $\mathbf{x}$ to the hyperplane: $\mathbf{x} = \mathbf{x}_p + r \frac{\mathbf{w}}{\|\mathbf{w}\|}$.
- $g(\mathbf{x}_p) = 0 \Rightarrow g(\mathbf{x}) = r\|\mathbf{w}\| \Leftrightarrow r = g(\mathbf{x})/\|\mathbf{w}\|$.

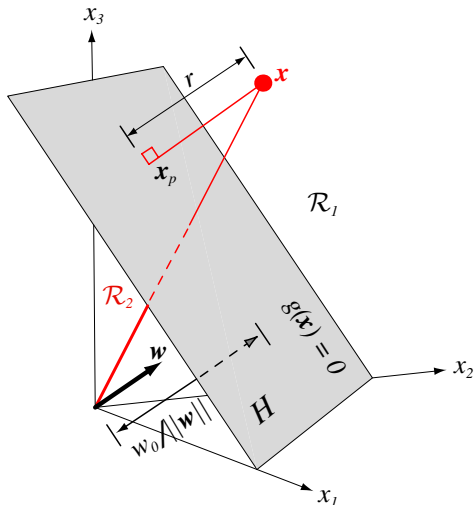


Fig 5.2 in Duda, Hart & Stork: Pattern Classification., Wiley 2001.

Generalized Linear Discriminant Functions

Use basis functions $\{b_1(\mathbf{x}), \dots, b_m(\mathbf{x})\}$, where each $b_i(\mathbf{x}) : \mathbb{R}^d \mapsto \mathbb{R}$, and $g(\mathbf{x}; \mathbf{w}) = w_0 + w_1 b_1(\mathbf{x}) + \dots + w_m b_m(\mathbf{x}) =: \mathbf{w}^t \mathbf{y}$ (note that we have redefined $\mathbf{y}$ here in order to be consistent with the following figure)

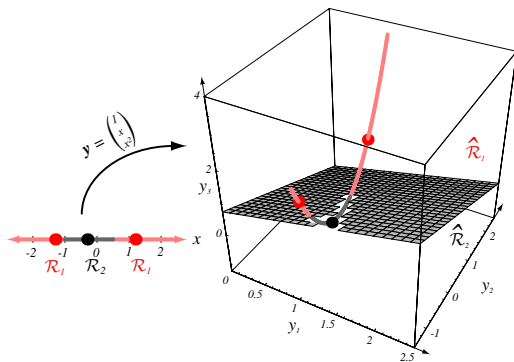


Fig 5.5 in Duda, Hart & Stork: Pattern Classification. Wiley 2001.

Generalized Linear Discriminant Functions

Use basis functions $\{b_1(\mathbf{x}), \dots, b_m(\mathbf{x})\}$, where each $b_i(\mathbf{x}) : \mathbb{R}^d \mapsto \mathbb{R}$, and

$$g(\mathbf{x}; \mathbf{w}) = w_0 + w_1 b_1(\mathbf{x}) + \dots + w_m b_m(\mathbf{x}) =: \mathbf{w}^t \mathbf{y}.$$

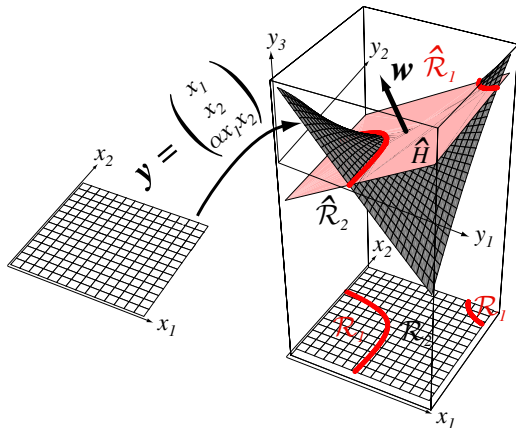


Fig 5.6 in Duda, Hart & Stork: Pattern Classification. Wiley 2001.

Separable Case

- Consider sample $\{\mathbf{y}_i, c_i\}_{i=1}^n$. If there exists $f(\mathbf{y}; \mathbf{w}) = \mathbf{y}^t \mathbf{w}$ which is positive for all examples in class 1 and negative for all examples in class 2, we say that the sample is **linearly separable**.
- Normalization:** replace all samples labeled c_2 by their negatives $\rightsquigarrow$ simply write $\mathbf{y}^t \mathbf{w} > 0$ for all samples.
- Each sample places a constraint on the possible location of $\mathbf{w}$
 $\rightsquigarrow$ **solution region**.

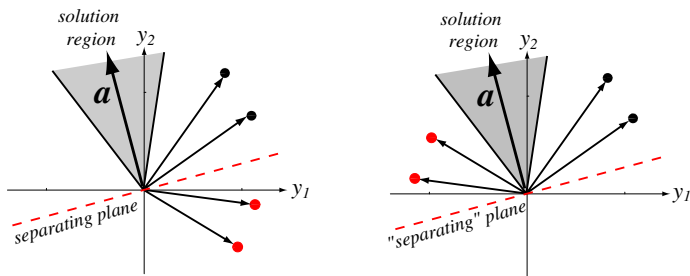


Fig 5.8 in Duda, Hart & Stork: Pattern Classification. Wiley 2001.

Separable Case: Margin

- Different solution vectors may have different **classification margins** b : $\mathbf{y}^t \mathbf{w} \geq b > 0$.
- Intuitively, **large margins are good**. We will formalize this in the chapter on **Statistical Learning Theory**.

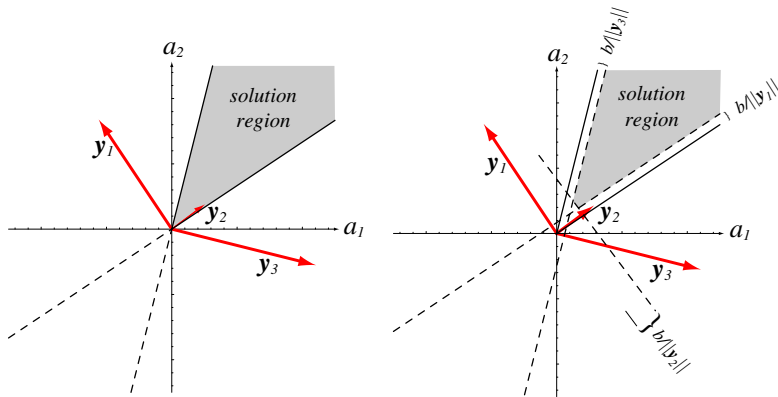


Fig 5.9 in Duda, Hart & Stork: Pattern Classification. Wiley 2001.

Gradient Descent

- Solve $\mathbf{y}^t \mathbf{w} > 0$ by defining $J(\mathbf{w})$ such that a minimizer of J is a solution.
- Most obvious choice for J : $\#(\text{misclassifications})$, **not differentiable.**
- Alternative choice: $J_p(\mathbf{w}) = \sum_{\mathbf{y} \in \mathcal{M}} -\mathbf{y}^t \mathbf{w}$, where $\mathcal{M}(\mathbf{w})$ is the set of samples **misclassified** by $\mathbf{w}$. **$J_p(\mathbf{w})$ is differentiable.**
- Note that $\mathbf{y}^t \mathbf{w} < 0 \forall \mathbf{y} \in \mathcal{M}$
 $\leadsto J_p$ **is non-negative, and zero only if $\mathbf{w}$ is a solution.**
- **Gradient descent:** Start with initial $\mathbf{w}^{(1)}$, choose next move in the direction of the negative gradient: $\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta^{(k)} \nabla J(\mathbf{w}^{(k)})$.
- Gradient: $\nabla J(\mathbf{w}) = - \sum_{\mathbf{y} \in \mathcal{M}} \mathbf{y}$. **Gradient descent:**
$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \eta^{(k)} \sum_{\mathbf{y} \in \mathcal{M}} \mathbf{y}.$$

This defines the **Batch Perceptron algorithm.**

Fixed-Increment Single Sample Perceptron

- Fix learning rate $\eta = 1$.
- Gradient step is **sum of individual steps** in the direction of **single misclassified samples**.
- **Sequential single-sample updates:** use superscripts $\mathbf{y}^1, \mathbf{y}^2, \dots$ for misclassified samples $\mathbf{y} \in \mathcal{M}$. Ordering is irrelevant.
- Simple algorithm:

$$\begin{aligned}\mathbf{w}^{(1)} & \text{ arbitrary} \\ \mathbf{w}^{(k+1)} &= \mathbf{w}^{(k)} + \mathbf{y}^k, \quad k \geq 1\end{aligned}$$

Perceptron Convergence Theorem

If the samples are linearly separable, the sequence of weight vectors given by the Fixed-Increment Single Sample Perceptron algorithm will terminate at a solution vector.

Proof: exercises.

Minimizing the Perceptron Criterion (2)

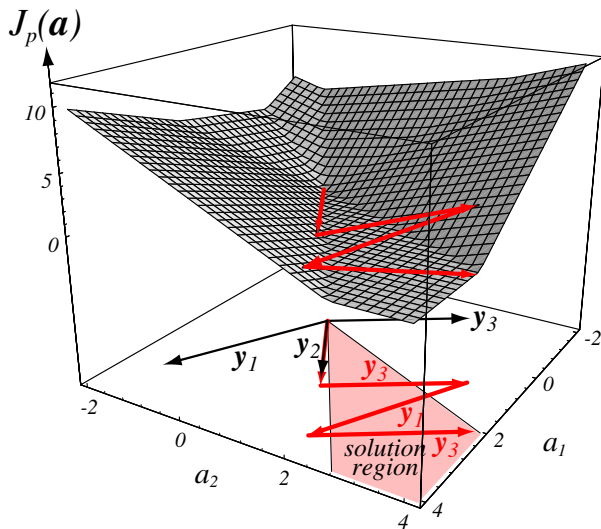


Fig 5.12 in Duda, Hart & Stork: Pattern Classification. Wiley 2001.

A number of problems with the perceptron algorithm:

- When the data are separable, there are **many solutions**, and which one is found **depends on the starting values**.
- In particular, no **separation margin** can be guaranteed (however, there exist modified versions...)
- The number of steps can be **very** large.
- When the data are **not separable**, the algorithm will **not necessarily converge**, and **cycles** may occur.
The cycles can be **long** and therefore **hard to detect**.
- Method “technical” in nature, no (obvious) probabilistic interpretation (but we will see that there is one).

But the perceptron algorithm is **historically important** (1957, one of the first ML algorithms!), was even implemented in analog hardware(!)

Generative (or Informative) vs Discriminative

- Notation: For the following discussion it is more convenient to go back to the original $\mathbf{x}$ -vectors (potentially after some basis expansion) instead of using the “normalized” representation $\mathbf{y}$.
- Two main strategies:
 - ▶ **Generative:** Generative classifiers specify how to generate data using the **class densities**.
Likelihood/posterior of each class is examined and classification is usually done by assigning to the most likely class.
 - ▶ **Discriminative:** These classifiers focus on modeling the **class boundaries** or the class membership probabilities directly.
No attempt is made to model the underlying class conditional densities.

Generative Classifiers

- Central idea: model the **conditional class densities** $p(\mathbf{x}|c)$.
- Assuming a parametrized family $p_{\mathbf{w}_j}(\mathbf{x}|c = j)$ and collecting all model parameters in a vector $\mathbf{w}$, a typical **Frequentist** approach now proceeds by maximizing the log likelihood:

$$\hat{\mathbf{w}}_{MLE} = \operatorname{argmax}_{\mathbf{w}} \sum_{i=1}^n \log p_{\mathbf{w}}(\mathbf{x}_i|c_i)$$

- (Approximate) **Bayesian** interpretation: $\hat{\mathbf{w}}_{MLE}$ might then be plugged into Bayes rule to compute the class assignment probabilities

$$P(c_j|\mathbf{x}) = \frac{p_{\hat{\mathbf{w}}_{MLE}}(\mathbf{x}|c = j)}{p(\mathbf{x})} P(c = j).$$

In **Linear Discriminant Analysis** (LDA), a Gaussian model is used where all classes share a common covariance matrix Σ :

$$p_{\mathbf{w}}(\mathbf{x}|c = j) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_j, \Sigma).$$

Generative Classifiers: LDA

- In **Linear Discriminant Analysis**, a Gaussian model is used where all classes share a common covariance matrix Σ :

$$p_{\mathbf{w}}(\mathbf{x}|c = j) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_j, \Sigma).$$

- The resulting **discriminant functions are linear**:

$$\begin{aligned} g(\mathbf{x}) &= \log \frac{P(c_1|\mathbf{x})}{P(c_2|\mathbf{x})} = \log \frac{P(c_1)\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_1, \Sigma)}{P(c_2)\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_2, \Sigma)} \\ &= \underbrace{\log \frac{P(c_1)}{P(c_2)} - \frac{1}{2} (\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2)^t \Sigma^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)}_{w_0} \\ &\quad + \underbrace{(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^t \Sigma^{-1} \mathbf{x}}_{\mathbf{w}^t \mathbf{x}} \\ &= w_0 + \mathbf{w}^t \mathbf{x}, \quad \text{with } \mathbf{w} = \Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2). \end{aligned}$$

LDA algorithm

- Let $\hat{\Sigma}$ be an estimate of the shared covariance matrix Σ :

$$\Sigma_c = \frac{1}{n_c} \sum_{\mathbf{x} \in \mathcal{X}_c} (\mathbf{x} - \mathbf{m}_c)(\mathbf{x} - \mathbf{m}_c)^t, \quad c \in \{c_1, c_2\}$$

$$\hat{\Sigma} = \frac{1}{2}(\Sigma_1 + \Sigma_2).$$

- Let $\mathbf{m}_j$ an estimate of μ_j :

$$\mathbf{m}_c = \frac{1}{n_c} \sum_{\mathbf{x} \in \mathcal{X}_c} \mathbf{x}, \quad n_c = |\mathcal{X}_c|.$$

- LDA finds the weight vector

$$\mathbf{w}^{\text{LDA}} = \hat{\Sigma}^{-1}(\mathbf{m}_1 - \mathbf{m}_2).$$

Law of large numbers: as $n \rightarrow \infty$, $\hat{\Sigma} \rightarrow \Sigma$ and $\mathbf{m}_j \rightarrow \mu_j$

$\leadsto$ This classifier **is asymptotically Bayes-optimal,**
if the Gaussian model with shared covariances is correct.

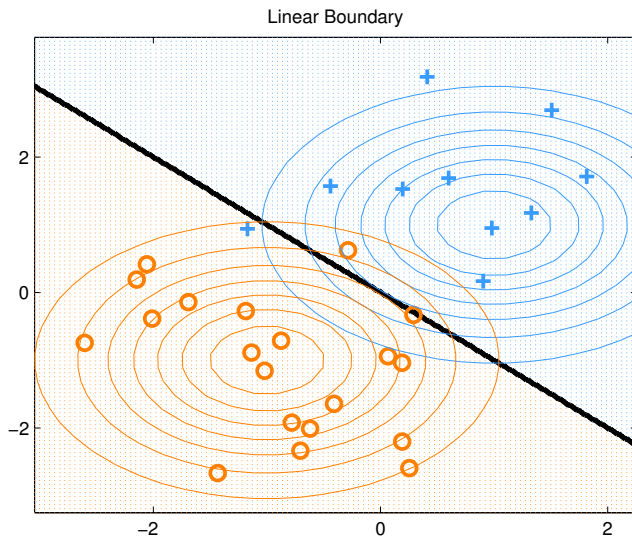


Fig 4.5 in K. Murphy: Machine Learning. MIT Press 2012

Discriminative classifiers

- Discriminative classifiers focus directly on the **discriminant function**.
- In general, they are more flexible with regard to the class conditional densities they are capable of modeling.
- Notation: Can use any class encoding scheme. Here: $c \in \{0, 1\}$.
- Bayes formula:

$$\begin{aligned} g(\mathbf{x}) &= \log \frac{P(c = 1|\mathbf{x})}{P(c = 0|\mathbf{x})} \\ &= \log \frac{p(\mathbf{x}|c = 1)P(c = 1)}{p(\mathbf{x}|c = 0)P(c = 0)}, \end{aligned}$$

- Can model any conditional probabilities that are exponential “tilts” of each other:

$$p(\mathbf{x}|c = 1) = e^{g(\mathbf{x})} p(\mathbf{x}|c = 0) \frac{P(c = 0)}{P(c = 1)}$$

Logistic Regression (LOGREG)

- **Logistic regression** uses a **linear discriminant function**, i.e. $g(\mathbf{x}) = \mathbf{w}^t \mathbf{x} + w_0$.
- For the special case $p(\mathbf{x}|c) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_{0,1}, \Sigma)$, same as LDA:

$$p(\mathbf{x}|c=1) = \mathcal{N}(\mathbf{x}, \boldsymbol{\mu}_1, \Sigma) = e^{g(\mathbf{x})} \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_0, \Sigma) \frac{P(c=0)}{P(c=1)}$$

$$\Rightarrow g(\mathbf{x}) = w_0 + \mathbf{w}^t \mathbf{x} = \log \frac{P(c=1)\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_1, \Sigma)}{P(c=0)\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_0, \Sigma)}$$

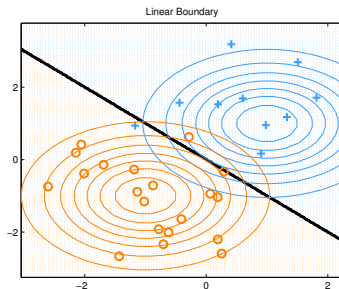
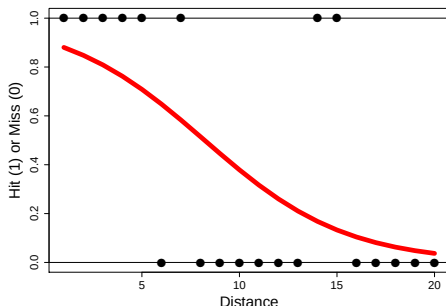


Fig 4.5 in K. Murphy: Machine Learning. MIT Press 2012

Logistic Regression (LOGREG)

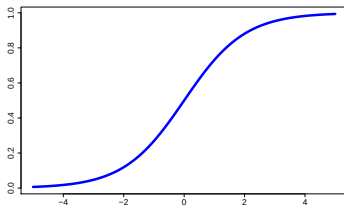
- Two-class problem with Bernoulli RV c taking values in $\{0, 1\}$
 $\rightsquigarrow$ sufficient to represent $P(1|\mathbf{x})$, since $P(0|\mathbf{x}) = 1 - P(1|\mathbf{x})$.
- “Success probability” of the Bernoulli RV: $\pi(\mathbf{x}) := P(1|\mathbf{x})$.
- Probability of miss ($c = 0$) or hit ($c = 1$) as a function of $\mathbf{x}$:
$$p(c|\mathbf{x}) = \pi(\mathbf{x})^c (1 - \pi(\mathbf{x}))^{1-c}, \quad \pi(\mathbf{x}) = P(c = 1|\mathbf{x}).$$
- Basketball example:



Adapted from Fig. 7.5.1 in B. Flury: A first course in multivariate statistics. Springer 1997.

Logistic Regression (LOGREG)

- LOGREG: $g(\mathbf{x}) = \mathbf{w}^t \mathbf{x} + w_0 = \log \frac{P(c=1|\mathbf{x})}{P(c=0|\mathbf{x})} = \log \frac{\pi(\mathbf{x})}{1-\pi(\mathbf{x})}$
- This implies $\frac{\pi(\mathbf{x})}{1-\pi(\mathbf{x})} = \exp\{g(\mathbf{x})\}$
 $\Rightarrow \pi(\mathbf{x}) = P(c=1|\mathbf{x}) = \frac{\exp\{g(\mathbf{x})\}}{1+\exp\{g(\mathbf{x})\}} =: \sigma(g(\mathbf{x})).$
- **Sigmoid** or **logistic** “squashing function” $\sigma(z) = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}$
turns linear predictions into probabilities



- Simple extension for K classes: the **softmax** function:
$$P(c=k|\mathbf{x}) = \frac{\exp\{g_k(\mathbf{x})\}}{\sum_{m=1}^K \exp\{g_m(\mathbf{x})\}}.$$

Logistic Regression (LOGREG)

- Assume that w_0 is “absorbed” in $\mathbf{w}$ using $\mathbf{x} \leftarrow (1, \mathbf{x})$. Estimate $\mathbf{w}$ by maximizing the conditional likelihood

$$\hat{\mathbf{w}}_{DISCR} = \operatorname{argmax}_{\mathbf{w}} \prod_{i=1}^n (\pi(\mathbf{x}_i; \mathbf{w}))^{c_i} (1 - \pi(\mathbf{x}_i; \mathbf{w}))^{1-c_i},$$

or by **minimizing the negative log likelihood:**

$$-l(\mathbf{w}) = - \sum_{i=1}^n [c_i \log \pi(\mathbf{x}_i; \mathbf{w}) + (1 - c_i) \log(1 - \pi(\mathbf{x}_i; \mathbf{w}))].$$

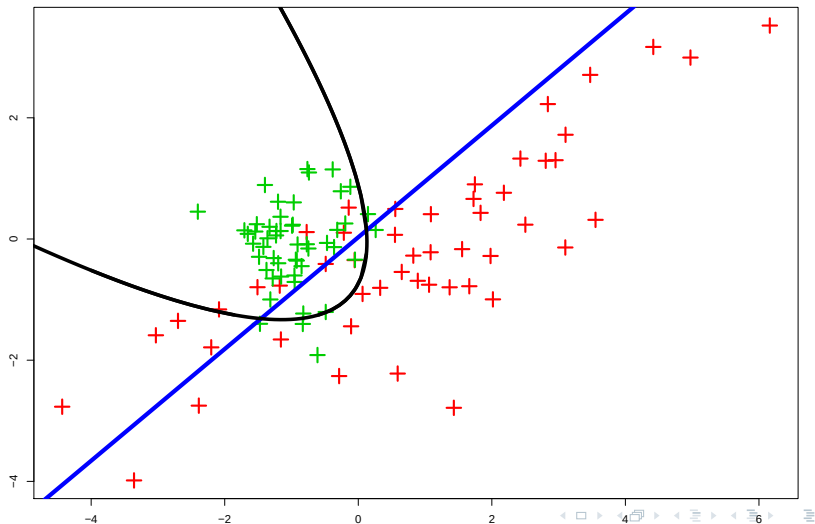
- The **gradient of l** is:

$$\nabla_{\mathbf{w}} l = \frac{\partial}{\partial \mathbf{w}} l(\mathbf{w}) = \sum_{i=1}^n \mathbf{x}_i (c_i - \pi_i).$$

- The π_i depend non-linearly on $\mathbf{w}$
 - $\rightsquigarrow$ equation system $\nabla_{\mathbf{w}} l = \mathbf{0}$ cannot be solved analytically
 - $\rightsquigarrow$ iterative techniques needed (e.g. gradient descent).

Logistic Regression (LOGREG)

Simple binary classification problem in $\mathbb{R}^2$. Solved with LOGREG using polynomial basis functions.

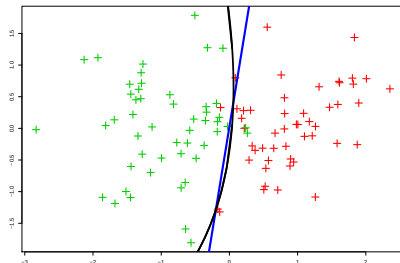


Logistic regression and BIC

Example: polynomial logistic regression, $n = 100$.

$\phi_1(\mathbf{x}) = (1, x_1, x_2)^t$, $\phi_2(\mathbf{x}) = (1, x_1, x_2, (x_1 + x_2)^2)^t$.

$$-\text{BIC} = \sum_{i=1}^n (-\log_2 p(y_i | \mathbf{x}_i, \hat{\mathbf{w}})) + \frac{d}{2} \log_2(n)$$



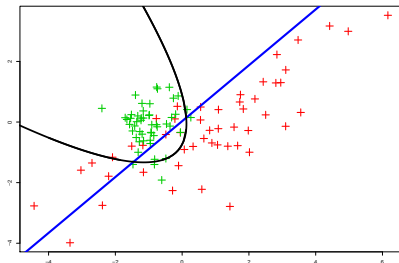
degree	#(param)	DL(data model)	DL(model)	BIC score
1	3	16.36 bits	9.97 bits	-26.33
2	4	15.77 bits	13.29 bits	-29.06

Logistic regression and BIC

Example: polynomial logistic regression, $n = 100$.

$\phi_1(\mathbf{x}) = (1, x_1, x_2)^t$, $\phi_2(\mathbf{x}) = (1, x_1, x_2, (x_1 + x_2)^2)^t$.

$$-\text{BIC} = \sum_{i=1}^n (-\log_2 p(y_i | \mathbf{x}_i, \hat{\mathbf{w}})) + \frac{d}{2} \log_2(n)$$



degree	#(param)	DL(data model)	DL(model)	BIC score
1	3	58.56 bits	9.97 bits	-68.53
2	4	38.05 bits	13.29 bits	-51.34

LOGREG and Perceptron

- Gradient of log-likelihood (at step k):

$$\nabla_{\mathbf{w}^{(k)}} l = \frac{\partial}{\partial \mathbf{w}} l(\mathbf{w})|_{\mathbf{w}=\mathbf{w}^{(k)}} = \sum_{i=1}^n \mathbf{x}_i (c_i - \pi_i^{(k)}).$$

- Gradient descent (for negative log.l.): $\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \eta \nabla_{\mathbf{w}^{(k)}}.$
- Assume stream of data $\rightsquigarrow$ online update for new observation $\mathbf{x}_i$:
$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \eta (c_i - \pi_i^{(k)}) \mathbf{x}_i, \quad \text{with } \pi_i^{(k)} = P(c = 1 | \mathbf{x}_i, \mathbf{w}^{(k)}).$$
- Now consider approximation: define **most probable label**
 $\hat{c}_i = \arg \max_{c \in \{0,1\}} P(c | \mathbf{x}_i, \mathbf{w}^{(k)})$ and replace π_i with $\hat{c}_i$.
- If we predicted correctly, then $\hat{c}_i = c_i \rightsquigarrow$ approximate gradient is zero
 $\rightsquigarrow$ update has no effect.
- If $\hat{c}_i = 0$ but $c_i = 1$: $\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \eta (c_i - \hat{c}_i) \mathbf{x}_i = \mathbf{w}^{(k)} + \eta \mathbf{x}_i.$
- Note that **this is again the perceptron algorithm.**
- Solution to most problems of the classical perceptron: use exact gradient instead of approximation based on most probable labels.
We do this in modern Multi Layer Perceptrons (MLP).

Loss function

- LOGREG minimizes the negative log likelihood

$$-l(\mathbf{w}) = - \sum_{i=1}^n [c_i \log \pi(\mathbf{x}_i; \mathbf{w}) + (1 - c_i) \log(1 - \pi(\mathbf{x}_i; \mathbf{w}))],$$

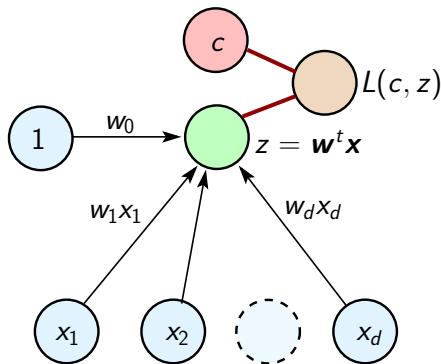
where $z = \mathbf{w}^t \mathbf{x}$, $\pi = \frac{1}{1+e^{-z}}$, $1 - \pi = \frac{e^{-z}}{1+e^{-z}}$.

- Introducing a **loss function**, we can write this as minimizing the average loss

$$\frac{1}{n} \sum_{i=1}^n \text{Loss}(c_i, z_i).$$

The perceptron / LOGREG

Can be viewed as an output layer in a neural network:



Adding additional layers $\rightsquigarrow$ **Multi-Layer Perceptrons (MLP).**